

1 Summary

This paper introduces computational techniques for sample-efficient evaluation of policy gradients in queueing systems, which enable policy optimization using orders of magnitude fewer samples than existing approaches. The authors introduce three techniques:

1. **Capacity sharing relaxation:** essentially, converting the problem into a continuous control problem by allowing fractional allocations of jobs to servers.
2. **Straight-through gradient estimation:** i.e., simulate with discrete dynamics, but compute gradients with respect to a softmax approximation to the actual dynamics.
3. **Work-conserving softmax,** a policy parameterization which prevents the policy from allocating server capacity to empty queues.

2 Overall recommendation

Overall, I think this is a strong paper that ultimately merits publication. While the paper’s theoretical contributions are light relative to the typical contribution to Operations Research, the practical contributions are interesting and practically meaningful. In particular, the problem of sample-efficient reinforcement learning for the real world, and in particular for operations and research problems, is under-explored in the literature relative to its importance. Some of the most successful approaches use differentiable simulators (e.g., hindsight optimization [MTE⁺22, ARK24]), but there is limited literature on such approaches outside of inventory control. This paper contributes to that literature significantly – the straight-through estimator is novel and highly effective, and queueing network control is traditionally a difficult domain for RL. The paper is well-written and the empirical evaluations are compelling.

3 Questions and comments

- **GPU parallelism** is mentioned early on in the abstract, but rarely comes up later in the manuscript. It seems like this is a feature of the simulator being well implemented, but is otherwise unrelated to the proposed methods. I think that 1) it should be made clear early on that GPU parallelizability is a separate idea from the other methods proposed; 2) the simulator should be made publicly available if possible, since this would be a very useful contribution in of itself; and 3) the authors may want to provide some measurements of their simulator’s wall-clock performance as a function of problem size, to make the utility of the simulator clear. Given that there are probably no existing GPU based queueing simulators to benchmark against, I think that a comparison against a CPU implementation would be

sufficient to make this point. An ideal experiment here would be to take a large-scale real-world queuing network control problem, and demonstrate the combined reduction in training time both from the implementation of the simulator as well as the sample efficiency gains from using the pathwise gradient. However, I don't have good suggestions on the choice of benchmark problem.

- **Ablation:** The proposed method actually introduces several changes relative to typical policy gradient approaches, which make it hard to tease out the real reason for the improvement: 1) The use of a continuous vs. discrete control policy; 2) The use of a deterministic rather than stochastic control policy; and 3) The use of first-order rather than zeroth-order gradients. The current experiments compare a discrete, stochastic policy with zeroth-order gradients (REINFORCE) against a continuous, deterministic policy with first-order gradients. The main additional ablation I would like to see (although I am not sure how straightforward this would be to construct) is the comparison of a continuous, stochastic policy (e.g., by adding Gaussian noise) with first-order gradients against a continuous, stochastic policy with zeroth-order gradients, in the spirit of [SSZT22]. One question is whether there are instances in which the zeroth order gradient estimator outperforms first order – for example, near discontinuities, although I'm not sure whether the queueing context changes the discussion at all.
- One of the key challenges of both PPO and REINFORCE is the sensitivity of the learned policy's performance to the **choice of hyperparameters**. While there's some discussion of this in the experiments assessing the quality of the Pathwise gradient estimator, it would be useful to see how sensitive the end-to-end policy optimization experiments are to the hyperparameters, in particular because I suspect that this will be a major strength of the proposed method relative to PPO / REINFORCE.
- I don't quite understand why **normalized gradient descent** is used in some of experiments, and relatedly, why cosine similarity is the primary metric used to judge the accuracy of gradient estimation (as opposed to Euclidean distance). As far as I'm aware, normalized SGD has convergence issues, and unnormalized SGD variants are much more common in practice – for which the magnitude of the estimated gradient will also need to be accurate.
- **Section 8:** I'm confused about how this section relates to the estimators considered in the rest of the paper. One issue is that it seems like A is sampled only once per trajectory, whereas in REINFORCE you would expect an independent draw at every timestep (although this probably means that the value of the first-order estimator is understated here). Regarding the pathwise estimator studied, is this equivalent to SmoothBackprop? It seems like there may be substantially more structure being exploited here. I understand that the overall goal is to show the value of first order versus zeroth order gradient estimators in a queueing-relevant context, but I'm not quite sure whether the intuition here is relevant to the actual implementation.
- **Notation / Typos**
 - I'm confused about the notation in Equation (5). The argument to each softmax should be a vector of length n corresponding to server i , so why are there subscripts j ? What is θx – aren't both of these vectors of length n ? is R still the routing matrix? Likewise in equation (8), is θ an $m \times n$ matrix or an n vector?
 - The summation in equation (Vanilla Softmax) overrides the j index
 - Page 33 line 3 seems to be missing part of the sentence

4 References

References

- [ARK24] Matias Alvo, Daniel Russo, and Yash Kanoria. Neural Inventory Control in Networks via Hindsight Differentiable Policy Optimization, April 2024.
- [MTE⁺22] Dhruv Madeka, Kari Torkkola, Carson Eisenach, Anna Luo, Dean P. Foster, and Sham M. Kakade. Deep Inventory Management, November 2022.
- [SSZT22] H. J. Terry Suh, Max Simchowitz, Kaiqing Zhang, and Russ Tedrake. Do Differentiable Simulators Give Better Policy Gradients?, August 2022.